

Q-SHIELD

Quantum-Resilient Self-Healing AIoT Security Architecture

COMPLETE PROJECT AUDIT

A brutally honest, evidence-first account of what has actually been built, what has been demonstrated in simulation, and what remains specification only — produced by inspecting the repository's source code and test suite as the sole source of truth, not its documentation's claims about itself.

544

TESTS PASSING, 0
FAILING

0

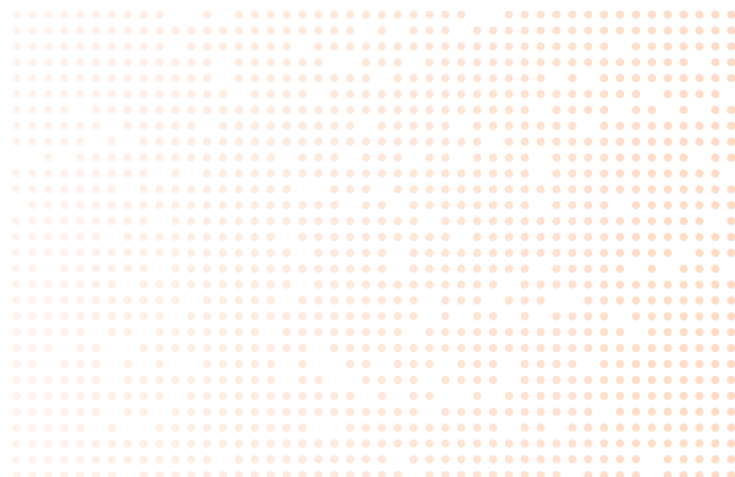
GIT COMMITS
(uncommitted tree)

12

MAJOR MODULES
AUDITED

VERIFIED = passing tests / reproducible artifact · PARTIAL = works, verification incomplete · SIMULATED = software stand-in, not real hardware · PLANNED = spec only, no working code

Prepared 29 September 2026 · Quant-A-Maze 3.0 · Track: Post-Quantum Cryptography



Contents

1. How to explain Q-SHIELD in 10 seconds, 60 seconds, and as an engineer
2. The problem, from zero
3. What Q-SHIELD is, and the architecture that actually exists
4. Phase-by-phase completion matrix
5. The trust engine, in depth (formula, factors, states, a real worked example)
6. Post-quantum cryptography, in depth
7. Vision / AI pipeline
8. Testing and validation
9. Performance — measured, not assumed
10. What is simulated vs. what is real
11. What is planned only — no working code
12. Limitations, in the project's own words
13. External research landscape and novelty analysis
14. Hackathon positioning
15. Final status board
16. Future implementation roadmap
17. Team pitches and the live-demo flow

1. How to explain Q-SHIELD

Q-SHIELD continuously re-evaluates whether an already-authenticated AIoT device can still be trusted, by fusing cryptographic identity, device-reported physical/sensor state, AI-observed visual evidence and network liveness into one deterministic, explainable 0–100 score — secured end-to-end by post-quantum cryptography (ML-KEM-768, ML-DSA-65) at the software layer.

10 SECONDS

“Authentication answers ‘who are you’ once. Q-SHIELD keeps asking ‘can I still trust you’, continuously, using post-quantum-signed evidence.”

60 SECONDS

An IoT device that authenticates once is trusted forever by most systems — even after it's compromised. Q-SHIELD's gateway authenticates every device message (HMAC today, designed with post-quantum migration in mind) and every AI observation (ML-DSA-65 signed, over an optional ML-KEM-768 encrypted session), then feeds only that authenticated evidence into a deterministic trust engine. The engine scores six factors — identity, physical, config integrity, sensors, vision and network — with hard caps so one confirmed incident can't be diluted by five healthy factors, and every score change comes with an exact, machine-readable reason. A real run pushed a device from TRUSTED (100) to SUSPICIOUS (73) using a genuine webcam+YOLO11n detection stream plus one forged and one replayed observation — both correctly rejected. Quarantine enforcement and self-healing recovery are specified and unit-tested at the state-machine level, but not yet wired to real access control.

TECHNICAL / JUDGE-LEVEL

Architecture: a FastAPI gateway terminates two independent authenticity domains — a per-device HMAC-SHA256 envelope protocol, and an ML-DSA-65 signed-observation protocol (optionally carried inside an ML-KEM-768-derived AES-256-GCM session) for a vision service. Both feed a pull-based trust service that turns gateway records into typed, authenticity-tagged Signals, applies them through a pure-logic engine (raw score = weighted average of available-factor penalties, renormalised over available weight; pressure term bounds unauthenticated input; hard caps gate confirmed incidents), and emits exactly-attributed TrustChange events. 544 tests pass across pqc, trust, security, integration and ai suites, including NIST ACVP known-answer vectors for ML-KEM-768/ML-DSA-65. What does not exist: an evidence hash-chain, a digital twin, a dashboard, an attack-simulation suite beyond one forgery/replay script, and any enforcement of the quarantine/recovery states the engine already validates.

2. The problem, from zero

WHAT IS THE PROBLEM?

Most IoT and AIoT security stops at the front door. A device proves who it is once — with a password, a certificate, a shared secret — and from that moment on, everything it says is treated as true. If the device is physically tampered with, its firmware is altered, its sensors are spoofed, or its credentials are stolen after that point, the system has no built-in way to notice. Authentication is a snapshot taken at one instant; trust is being asked to last forever from that snapshot.

WHY DOES IT MATTER?

AIoT devices increasingly make decisions or feed decisions that affect physical safety — industrial sensors, healthcare monitors, building security cameras. A compromised device that is still “trusted” because it once authenticated correctly can silently feed false data, hide a real event, or become a foothold for a larger attack — all while every log says “authenticated: yes.”

WHY ARE CURRENT IoT SECURITY SYSTEMS INSUFFICIENT?

Typical IoT security stacks (TLS/DTLS, certificate-based device identity, static access-control lists) answer “is this device who it claims to be, right now, at the network layer.” They generally do not: (a) fuse physical, sensor, visual and network evidence into one continuous judgement of trustworthiness; (b) explain exactly why a trust judgement changed; (c) survive the arrival of a cryptographically-relevant quantum computer, which would break the RSA/ECC identity most such systems still rely on.

WHAT IS Q-SHIELD, AND WHAT MAKES IT DIFFERENT?

Q-SHIELD is a prototype gateway architecture that keeps authentication and trust as two separate, explicit concepts. A device or vision service must still authenticate every message — with a symmetric HMAC secret for the ESP32 today, and with NIST-standardised post-quantum ML-DSA-65 signatures (optionally over an ML-KEM-768 session) for the vision layer. But authenticating never, by itself, raises trust. Trust is a separate, continuously-recomputed score derived only from what the authenticated evidence actually says: is the tamper switch reporting open, are sensor readings in range, does the vision pipeline see something the policy flags, is the device still reachable. A valid signature can only ever gate which evidence is allowed to count — never add points on its own.

HOW DOES IT WORK, END TO END?

Device → authenticated telemetry → gateway → trust engine reads the authenticated record → factor penalties update → a deterministic 0–100 score and a state (TRUSTED / SUSPICIOUS / QUARANTINED / RECOVERING / VERIFIED / RECOVERED) are published with an exact, itemised reason for the change. Section 3 gives the full pipeline diagram, separating what is real from what is only specified.

WHAT HAPPENS WHEN A DEVICE IS COMPROMISED, AND HOW DOES RECOVERY WORK?

The trust engine will drop the score, apply a hard cap (so, for example, a confirmed physical-plus-visual incident is capped at 30 regardless of the weighted average) and record QUARANTINED as the state with a recommended action. In the current prototype that action is a recommendation only — nothing in the gateway today actually revokes a session or blocks an endpoint because of it (see §11). The state machine also defines a validated recovery path — RECOVERING → VERIFIED → RECOVERED — gated on fresh authenticated evidence and score thresholds, and unit-tested, but nothing drives a real device through it yet.

WHY DOES POST-QUANTUM CRYPTOGRAPHY MATTER HERE?

A sufficiently capable quantum computer would break the RSA/ECC signatures and key exchange that almost all deployed IoT identity systems use today, via Shor's algorithm. Data or identities protected only by classical

public-key cryptography can be harvested now and decrypted or forged later once such a computer exists (“harvest now, decrypt later”). Q-SHIELD’s vision-service identity layer is built on ML-KEM-768 and ML-DSA-65 — the NIST FIPS 203/204 post-quantum standards — from the start, rather than as a later retrofit.

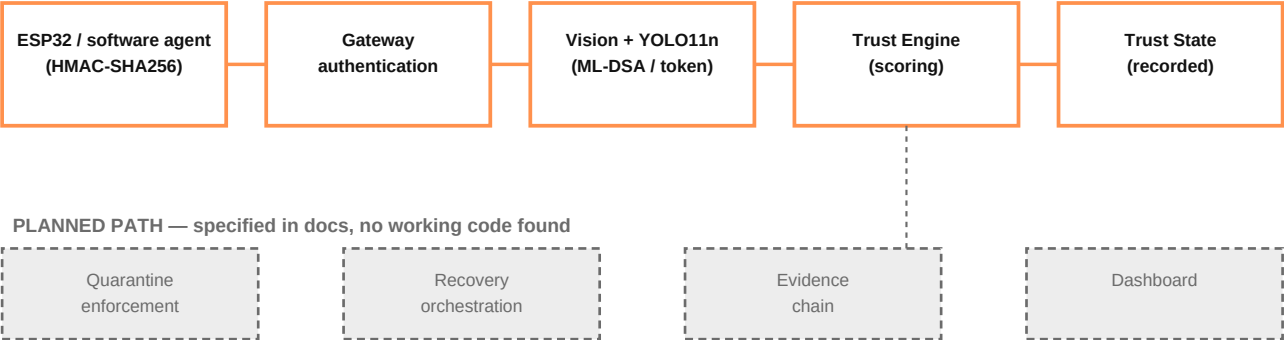
THE ROLE OF AI, VISION, THE TRUST ENGINE, THE DIGITAL TWIN AND THE EVIDENCE CHAIN

AI / vision (real, tested): a YOLO11n detector over a live webcam produces visual observations, which are signed and become one input to the trust score — never a verdict by themselves. **Trust engine** (real, tested, the most mature module): the deterministic scorer described in §5. **Digital twin** (planned only): the architecture documents describe comparing a device’s self-reported firmware/config hash against an expected “known-good” twin state; no such comparison module exists in the code — only two optional configuration fields the trust engine can use if a deployer fills them in by hand. **Evidence chain** (planned only): an append-only, cryptographically-chained forensic log is specified (TD-09) but not implemented anywhere in the repository.

3. What Q-SHIELD is, and the architecture that actually exists

The diagram below is deliberately split in two: the top row is the path that is implemented, tested and has a reproducible live-run artifact behind every box. The bottom row is the response/forensics path that is specified in the architecture documents but has no working code — shown in gray, dashed, to keep the report honest about where the real system currently ends.

REAL PATH — implemented, tested, has a live-run artifact



Communication paths, precisely:

Authentication flow	Device → HMAC-SHA256 envelope → gateway authenticate(); Vision service → ML-DSA-65 signature (+ optional ML-KEM-768 session) → gateway verify_signed_observation(). Two independent domains; an operator bearer token and an ingest bearer token are two more, and none of the four are interchangeable.
Data flow	Authenticated telemetry and observations are stored as gateway records (device_messages, observations, security_events) — the trust engine only ever reads these records, never the wire traffic directly, keeping trust computation out of the detector, camera, firmware and crypto code.
Trust flow	A pull-based TrustService converts new records into typed Signals (backend/trust/adapters.py), applies them through the pure TrustEngine, and persists an exact TrustChange event per material change (§5).
Attack flow (demonstrated)	scripts/demo_forgery.py takes a real stored signed observation, alters its payload (signature no longer matches — rejected 401) and replays the original unchanged (rejected 409 inside the freshness window). Both rejections feed the trust engine as bounded, unauthenticated “pressure”.
Quarantine / recovery flow (specified, not enforced)	The state machine validates the full TRUSTED→SUSPICIOUS→QUARANTINED→RECOVERING→VERIFIED→RECOVERED graph and is unit-tested, but no code path in the gateway currently revokes a session, blocks an endpoint, or drives a device through remediation.
Evidence flow (specified, not implemented)	TD-09 designs a SHA-256 hash chain of ML-DSA-signed entries; no such structure exists in the code today.

4. Phase-by-phase completion matrix

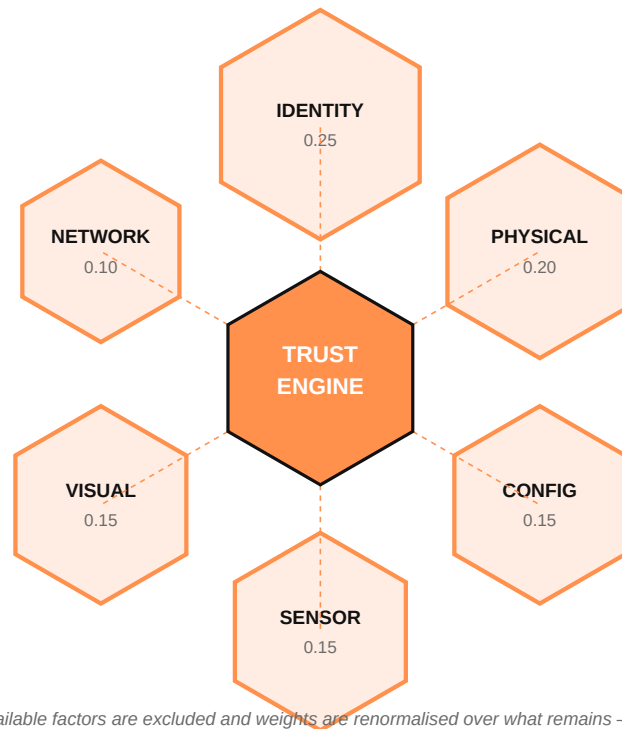
Q-SHIELD has zero git commits — the entire repository is one uncommitted working tree, so this timeline is reconstructed from the phase markers the project's own documentation uses internally, not from version-control history.

Phase	Feature	Status	Evidence	Remaining
0-1	Foundation · Device ↔ Gateway	verified	HMAC-SHA256 envelope, register/heartbeat/telemetry, replay/counter protection, operator+ingest tokens	ESP32 firmware not compiled/flushed
2	Vision + AI	verified	YOLO11n over real webcam, zones, camera health, 108 tests, live-run-phase3.json	Detection accuracy not measured; restricted-zone path not exercised live
3	Post-Quantum Cryptography	verified	ML-KEM-768 + ML-DSA-65 via pqcrypto 1.0.0, ACVP KAT vectors, AES-256-GCM sessions, 74+135 tests	Session protocol has had no external review; ESP32 stays HMAC-only
4	Dynamic Trust Engine	verified	Deterministic scorer, 197 tests, exact explainability, trust_bench.json, live trust run to SUSPICIOUS	Parameters are design choices, not calibrated (\$5.7)
4.1	Trust Engine Review	verified	Fixed a real design flaw (RECOVERING auto-regression), added score guards on explicit transitions, terminology fix	Same parameter-tuning caveat carries forward
5	Attack Simulation	partial	One real scenario (forge + replay) in scripts/demo_forger.py	No suite; tamper/sensor/config-integrity/rogue-device scenarios not scripted
6	Quarantine	planned	State recorded and validated; RECOMMENDED_ACTION computed	Nothing blocks an endpoint or revokes a session because of it
7	Self-Healing / Recovery	planned	State-machine transitions unit-tested (recovery_fault, score guards)	No orchestrator drives a device through remediation; tests/recovery/ is empty
8	Evidence Chain	planned	TD-09 design only	evidence/README.md is 0 bytes; no code anywhere
9	Digital Twin	planned	Two optional config fields the trust engine can consume	No comparison module; no twin state store
10	Dashboard	planned	TD-14 design only	dashboard/README.md is 0 bytes; no frontend files exist
11	Final Integrated Demo	planned	—	Depends on Phases 6-10

5. The trust engine, in depth

This is the most mature part of the project — pure logic, 197 passing tests, and a specification (section 4.7 of docs/architecture/trust-engine.md) that is normative: the code is checked against it directly.

5.1 The six factors



Hexagon size = nominal weight. Unavailable factors are excluded and weights are renormalised over what remains — never assumed healthy.

Each factor carries a penalty from 0 (no adverse evidence) to 100 (worst). A factor with no evidence yet is *unavailable* — excluded from the score entirely and reported as such, never assumed healthy. This is a deliberate design choice with a real consequence: a device that has only ever sent clean authenticated telemetry (identity + network available, everything else unavailable) scores 100 at 35% coverage, and the report says so explicitly — the score reflects only the evidence that exists.

5.2 The exact formula

```

raw = 100 - sum( w'_i * p_i ) over available factors i, where w'_i = w_i / sum(available w_i)
uncapped = clamp(raw - pressure, 0, 100)
final = min(uncapped, every active cap ceiling)
score = round_half_up(final) -- an integer in [0, 100], published; the exact value is kept internally
  
```

Weights are re-normalised over available factors, never assumed. Confidence scales visual impact ($100 \cdot c^2 \cdot m_{\text{auth}}$, floor 0.30, halved for token-only observations) but can never override the authenticity gate — an unauthenticated source cannot deliver a visual signal at all. “Pressure” is a separate, capped (≤ 25) term for unauthenticated network noise (forged signatures, replays): bounded so a forged flood can push a device to SUSPICIOUS at most, never QUARANTINED.

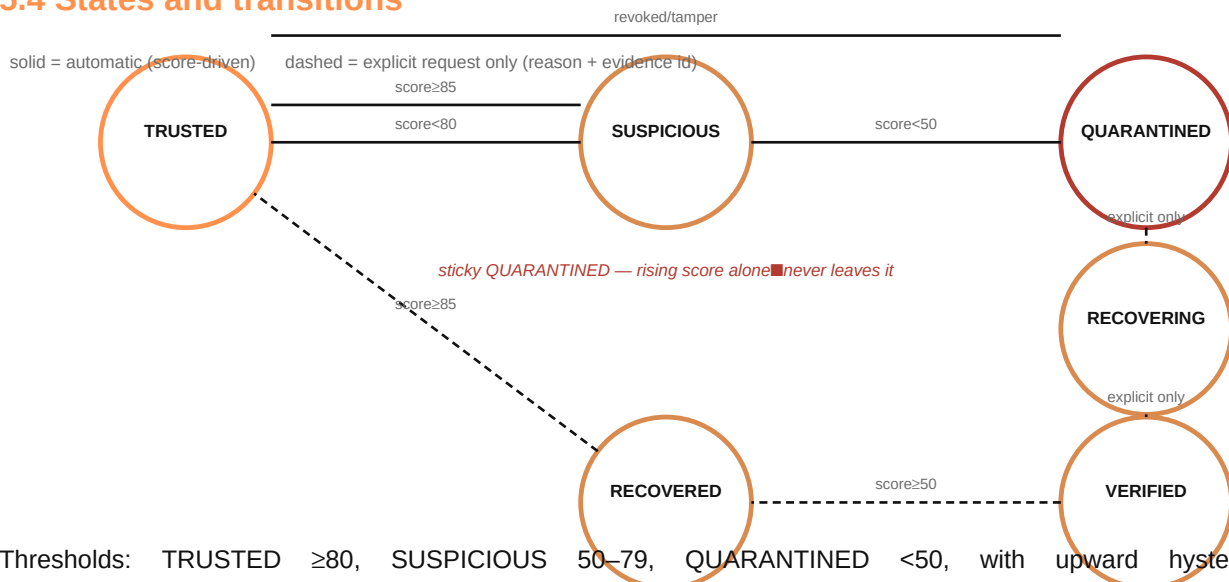
Factor	Weight	What sets its penalty
identity_crypto	0.25	auth_misbehavior (60), malformed_payload (30) — from an authenticated sender
physical	0.20	physical_tamper (100, CRITICAL) — device-reported tamper switch, once per episode
config_integrity	0.15	integrity_mismatch (60) — self-reported fw/config hash vs. configured expectation
sensor_consistency	0.15	sensor_out_of_range (60) — telemetry field outside a configured limit
visual	0.15	visual_rule_violation ($100 \cdot c^2 \cdot m_{\text{auth}}$), camera_obstructed (60), camera_source_lost (30)

Factor	Weight	What sets its penalty
network	0.10	not persistent — pure function of time since the last authenticated device message

5.3 Caps — the reason one bad signal cannot be diluted by five good ones

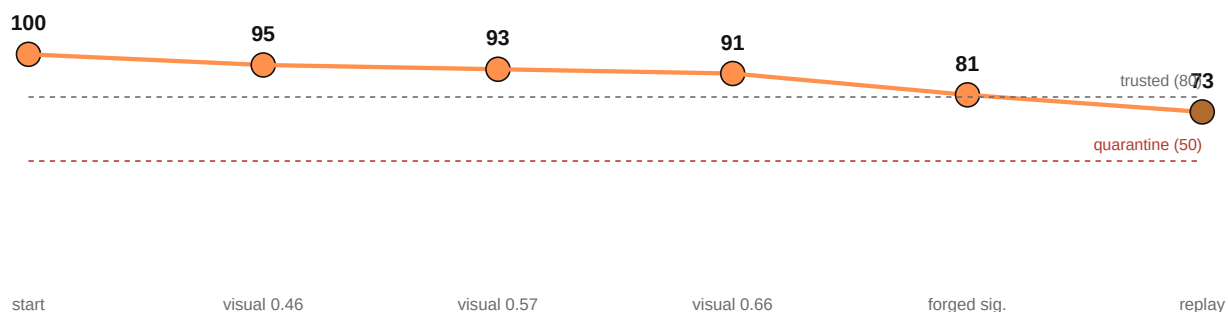
Cap	Ceiling	Condition
revoked_device	0	device credential revoked
confirmed_incident	30	tamper + ≥ 1 other modality within 60s (correlation window)
physical_tamper	55	tamper reported, unconfirmed by any other modality
correlated_incident	55	≥ 2 modalities without tamper
repeated_replay	65	≥ 3 replay events within 600s
integrity_mismatch	65	latest self-reported config/firmware hash mismatches expectation
auth_violation	70	a valid signer misbehaves (wrong device/metadata)
stale_device	79	no authenticated device message for >45s

5.4 States and transitions



Thresholds: TRUSTED ≥ 80 , SUSPICIOUS 50–79, QUARANTINED < 50 , with upward hysteresis (SUSPICIOUS $\rightarrow$ TRUSTED needs ≥ 85) to avoid flapping at the boundary. QUARANTINED is sticky by design — TD-10's own reasoning is quoted verbatim in the spec: a reboot or a period of silence is not recovery. The Phase 4.1 review fixed a genuine design flaw here: the original rule dropped RECOVERING/VERIFIED back to QUARANTINED automatically whenever the score was below 50 — but a device fresh out of quarantine is *always* below 50, so that rule would have made recovery structurally impossible. It now holds those states until either revocation or a fresh *authenticated* report that the fault is still present — explicitly excluding unauthenticated network noise, which must never be able to knock a device out of recovery.

5.5 A real worked example (from the actual Phase 4 live run)



DEVICE-001 authenticated via the software agent $\rightarrow$ score 100, TRUSTED. Three genuine YOLO11n detections over a live webcam, signed ML-DSA-65, at confidences 0.46 / 0.57 / 0.66 $\rightarrow$ 100 $\rightarrow$ 95 $\rightarrow$ 93 $\rightarrow$ 91, still TRUSTED. A controlled forged signature (scripts/demo_forger.py) was correctly rejected by the gateway and fed the engine's bounded pressure term $\rightarrow$ 91 $\rightarrow$ 81. A replay of the original signed observation, also correctly rejected, added more pressure $\rightarrow$ 81 $\rightarrow$ 73, crossing into SUSPICIOUS. No physical tamper was performed or fabricated for this run — the live-run artifact says so explicitly.

5.6 Plain-English version

Start at 100 $\rightarrow$ an authentication violation or a forged message nudges the score down a little, bounded so it alone can't cause quarantine $\rightarrow$ a real visual rule violation lowers a different factor $\rightarrow$ the score keeps falling and the state flips to SUSPICIOUS $\rightarrow$ if a second, independent kind of evidence corroborates the first (say, tamper plus vision) within the correlation window, a hard cap forces QUARANTINED regardless of what the weighted average says — confirmed physical-plus-visual evidence is never diluted by otherwise-healthy factors. Recovery only ever happens through fresh, clean, *authenticated* evidence accumulated over real elapsed time ("credited seconds") — silence, or an attacker simply stopping, credits nothing and heals nothing.

5.7 Design choice vs. measured vs. needs tuning

The specification is unusually candid about this, and this report preserves that honesty rather than upgrading it: "Nothing in this repository shows that any weight, threshold, half-life, floor, cap or point value was optimised, calibrated or validated: there is no incident dataset, no false-positive/false-negative study, no ROC analysis, no user study." Its own four-way split:

Category	Examples	Status
A — established project decisions	factor weights, 80/50 thresholds (TD-08)	Decided, not empirically derived
B — measured results	trust_bench.json timings; one live run	Real numbers, on one laptop, one scenario
C — prototype design choices	severity scale, visual exponent, all half-lives, caps	Author's choice, unvalidated
D — needs empirical tuning	confidence floor/exponent, pressure points, coverage policy	Open for review before a real deployment

6. Post-quantum cryptography, in depth

WHAT PQC IS, AND WHY Q-SHIELD USES IT

Post-quantum cryptography (PQC) is cryptography designed to resist attacks by a large-scale quantum computer running Shor's algorithm, which would break RSA and elliptic-curve cryptography — the identity systems almost every IoT deployment relies on today. NIST finalised its first PQC standards in 2024 (ML-KEM, FIPS 203; ML-DSA, FIPS 204; SLH-DSA, FIPS 205), with a fifth backup KEM (HQC) selected in March 2025. Q-SHIELD's vision-service identity layer is built on ML-KEM-768 and ML-DSA-65 — the exact FIPS 203/204 parameter sets — from the start.

WHAT ACTUALLY USES PQC, AND WHAT DOES NOT

Uses PQC: the vision service's identity and every signed observation it sends (ML-DSA-65 signature; optional ML-KEM-768-derived AES-256-GCM session for confidentiality). **Does not use PQC — explicitly, by design, documented in the project's own decision record (TD-01):** the ESP32 device path, which authenticates with a symmetric HMAC-SHA256 credential. TD-01 lists the claims the project may not make: “that the ESP32 performs PQC; that ESP32 telemetry is ‘PQC-protected’; ‘quantum-proof’.” This report repeats that constraint deliberately — the ESP32 is not, and has never been claimed to be, post-quantum secured.

EXACT PARAMETERS AND LIBRARY

Item	Value
Key encapsulation	ML-KEM-768 (FIPS 203)
Signatures	ML-DSA-65 (FIPS 204)
Library	pqcrypto 1.0.0 (PyPI, Apache-2.0, Rust-backed prebuilt wheel)
Session cipher	AES-256-GCM
Key derivation	HKDF-SHA256 — separate info labels for session keys and for keystore key-encryption-keys
Replay / timestamp protection	±300s clock-skew window on signed observations; strictly-increasing per-direction 64-bit counters as AES-GCM nonces; a seen-nonce cache blocks handshake replay
Key storage	Private keys sealed with AES-256-GCM under HKDF subkeys of a master key; round-trip self-test on load
Provisioning / lifecycle	scripts/pqc_provision.py, scripts/enroll_device.py; no automated rotation or revocation beyond re-provisioning — documented as “not an HSM/TPM”
Standards conformance	NIST ACVP known-answer-test vectors pass for decapsulation, key-validity checks and signature verification
ESP32 path	HMAC-SHA256 only — never PQC (TD-01; backend/security/auth.py explicitly refuses the PQC auth profile rather than pretend to verify it)

The session protocol itself — ML-KEM key establishment, HKDF derivation, AES-256-GCM framing — is a custom composition of standard primitives, not an off-the-shelf protocol like TLS. The module's own docstring is explicit about the resulting limitation: “NOT a vetted protocol... has had no external review” and does not provide forward secrecy for the gateway's static KEM key beyond ML-KEM's own properties, session resumption, or response confidentiality.

7. Vision / AI pipeline

Item	Value
Model	YOLO11n (Ultralytics)
Licensing note	Ultralytics weights are AGPL-3.0; the project's own decision record flags that distributing or network-serving a combined work may carry AGPL obligations that have not had legal review
Inference framework	Ultralytics/PyTorch, CPU, imgsz 640
Measured inference latency	39.8ms mean at imgsz 640 (~25.1 FPS inference-only, ~21.0 FPS end to end) — a single-run measurement on one laptop
Detection accuracy	NOT MEASURED — the project's own decision record states no accuracy is claimed until measured on recorded data with a documented method
Confidence handling	Passed through to the trust engine as the magnitude scale for a visual rule violation (§5.2); never a pass/fail verdict on its own
Signing / gateway integration	Observations are signed ML-DSA-65 (or carried token-only) and ingested through the same authenticity boundary as every other trust signal
Trust engine integration	Real, tested — see the worked example in §5.5

Inference FPS (how fast the model alone runs) is not the same number as end-to-end system FPS (capture + inference + zone logic + signing + network send), and neither is a measurement of detection accuracy (whether the boxes are actually right). This report keeps the three separate because the project's own docs were careful to — conflating them is one of the easiest ways to overstate a vision system.

The one real live run (docs/results/live-run-phase3.json) used a genuine USB webcam and produced 11 stored, ML-DSA-65-signed observations of real objects in frame (8 “chair”, 3 “potted plant”, confidence range 0.318–0.618). All 11 were independently re-verified offline; a tampered copy of each was correctly rejected. No person was in view during that run, so the restricted-zone / person-detection path — the scenario most relevant to a security demo — has not itself been exercised live, only exercised in the later Phase 4 trust run (§5.5), which did detect real objects flagged by the configured rule.

8. Testing and validation

544 tests pass, 0 fail (python -m pytest, full suite, 38.5s). Three test directories exist but are empty (only `__init__.py` or fixture data) — listed honestly rather than omitted.

Directory	Tests	What it actually checks
tests/trust	197	Score equation vs. an independent re-implementation, authenticity boundaries, confidence handling, caps/fusion, temporal decay, state-machine legality, recovery semantics (Phase 4.1), explainability sum-to-delta, adversarial/malformed input, determinism
tests/security	135	HMAC envelope auth, replay/counter protection, credential storage, gateway authentication failures
tests/pqc	74	ML-KEM-768/ML-DSA-65 correctness, NIST ACVP known-answer vectors, signed-envelope verification, session establishment/replay
tests/ai	108	Detector wrapper, pipeline, zones, camera health, sinks/runner, observation schema
tests/integration	30	End-to-end HTTP flows across the gateway
tests/end_to_end	0	Empty — only <code>__init__.py</code>
tests/recovery	0	Empty — only <code>__init__.py</code>
tests/vectors	0	Fixture data only (ACVP JSON vectors, envelope schema)

Capability	Test	Result	Evidence
PQC correctness	NIST ACVP known-answer vectors (ML-KEM-768, ML-DSA-65)	PASS	tests/pqc/test_conformance_acvp.py
Signature forgery rejection	Live gateway, altered signed observation	PASS (401)	docs/results/live-run-phase3.json, trust-live-run-phase4.json
Replay rejection	Live gateway, exact replay within freshness window	PASS (409)	same, plus scripts/demo_forgery.py
Vision pipeline	Real USB webcam + YOLO11n	PASS (mechanism)	live-run-phase3.json
Trust scoring determinism	Independent re-implementation of the formula	PASS	tests/trust/test_spec_equation.py
Recovery-state semantics	Unit tests for recovery_fault / score guards	PASS	tests/trust/test_recovery_semantics.py
Physical tamper detection (real hardware)	—	NOT RUN	ESP32 never compiled/flashed
Digital twin comparison	—	NOT APPLICABLE	No module exists

9. Performance — measured, not assumed

Every number below is quoted from docs/results/*.json, produced by scripts/bench_pqc.py and scripts/bench_trust.py. All are laptop measurements on one machine (Windows 11, i7-13700HX, Python 3.12.0) and are explicitly not real-time or deployment-latency claims — the JSON files say so themselves.

Operation	Mean	Notes
ML-KEM-768 key generation	0.061 ms	n=1500
ML-KEM-768 encapsulate	0.073 ms	
ML-KEM-768 decapsulate	0.095 ms	
ML-DSA-65 key generation	0.240 ms	
ML-DSA-65 sign	9.413 ms	p95 = 10.146 ms
ML-DSA-65 verify	0.204 ms	
Full session handshake	10.060 ms	n=300, p95 = 10.798 ms
Trust engine apply() (single signal)	52.6 µs	n=20000, p99 = 73.2 µs
Trust service, incremental process	742.99 µs	n=300, p99 = 1330.8 µs
Trust service, batch throughput	6,723 rows/s	2100 rows in 0.312s
Vision inference (imgsz 640)	39.8 ms	~25.1 FPS inference-only, ~21.0 FPS end-to-end

10. What is simulated vs. what is real

Claim	Reality
"Device telemetry"	Produced by device_agent (backend software), labelled hw=software-agent throughout. It replays the real ESP32 HMAC wire protocol over HTTP so the gateway can be exercised without hardware — the live-run artifact itself states this explicitly: "SIMULATED, not the ESP32."
"Physical tamper detection"	The tamper signal path is real code with real tests, but no physical tamper was ever performed or fabricated in any live run — stated explicitly in trust-live-run-phase4.json.
"ESP32 hardware"	hardware/esp32/firmware/src/main.cpp carries its own header comment: "STATUS: NOT COMPILED, NOT FLASHED, NOT TESTED. Written without a connected board." It deliberately does not send telemetry, because "reporting tamper=false without a real reed switch would be a false claim."
"Vision / AI"	Real: a genuine USB webcam and YOLO11n model, real detections, real ML-DSA-65 signatures, real offline re-verification. Detection accuracy is not measured.
"Post-quantum cryptography"	Real: actual ML-KEM-768/ML-DSA-65 operations via the pqcrypto library, passing NIST ACVP known-answer vectors — but only in the vision-service path, never on the ESP32.
"Attack simulation"	One real, reproducible scenario (forgery + replay) via scripts/demo_forgery.py, run against the live gateway. Not a suite; not a claim of comprehensive attack coverage.
"544 tests passing"	Real, reproducible: python -m pytest, 0 failures, verified during this audit.
"Trust score dropped to SUSPICIOUS after an attack"	Real: a live HTTP run against the actual gateway process, not a mocked unit test — docs/results/trust-live-run-phase4.json.

11. What is planned only — no working code

For each item below, a repository-wide search was performed for its concept name; every result was a documentation or slide mention, never an implementation.

Item	Justification
Evidence / hash chain	evidence/README.md is a tracked file with 0 bytes. TD-09 describes the intended design (SHA-256 chained entries, ML-DSA-signed) in full, but no module anywhere implements an append-only chain.
Digital twin	backend/trust/config.py has two optional fields (expected_fw_version, expected_cfg_hash) the trust engine can consume if a deployer configures them by hand — the spec itself calls this “configuration-derived expectations (no digital twin yet).” There is no comparison module, no twin state store, no automatic derivation of the expected state.
Dashboard	dashboard/README.md is 0 bytes. No dashboard/ source directory, no package.json, no frontend framework files exist anywhere in the repository. TD-14 specifies a Vite+React+TypeScript stack, explicitly deferred to “Phase 10.”
Quarantine enforcement	state_machine.py's own docstring: “Phase 4 records states and validates transitions. It does NOT enforce quarantine or drive recovery.” The recommended-action string for QUARANTINED literally reads “quarantine recommended (not enforced in Phase 4).”
Recovery orchestration	The state machine's recovery_fault flag and score guards (Phase 4.1) are real and unit-tested, but nothing drives a device through TD-10's designed sequence (revoke → freeze evidence → safe mode → compare to twin → push known-good config → health checks). tests/recovery/ contains zero tests.
Attack-simulation suite	attack_simulation/README.md is 0 bytes; no code exists under that directory. The only real attack code is the single scripts/demo_forgery.py scenario.
docs/architecture/self-healing.md, security-architecture.md	Both are tracked files with 0 bytes.

12. Limitations, in the project's own words

Every quotation below is verbatim from the repository's own documentation — nothing here is this report's editorializing.

"Composition of standard primitives, NOT a vetted protocol (it is not TLS and has had no external review)." — backend/security/session.py

"Not provided: forward secrecy for the gateway's static KEM key beyond ML-KEM's own properties... session resumption, or response confidentiality." — backend/security/session.py

"Claims we may not make: that the ESP32 performs PQC; that ESP32 telemetry is 'PQC-protected'; 'quantum-proof'." — TD-01

"Vendor is small; independent audit status not verified... No FIPS 140 validation is claimed, ever, for this prototype." — TD-02

"Phase 1 provides integrity/authenticity/replay protection, not confidentiality... We do not add TLS on the ESP32 in v1." — TD-03

"An attacker with both DB write access and the gateway signing key can rewrite history; deleting the tail is detectable only if the latest head hash is anchored somewhere external... true external anchoring is out of scope." — TD-09 (design document for an unbuilt module)

"Firmware/config hashes are self-reported by a possibly compromised device; without ESP32 secure boot + remote attestation they are evidence, not proof." — TD-10

"No detection accuracy is claimed until measured on recorded data with a documented method." — TD-11

"Distributing a combined work or serving it over a network may carry AGPL obligations; this has not had legal review." — TD-11 (Ultralytics/YOLO licensing)

"Does not protect against: an attacker with both the DB and keys/ (same host/user), a compromised gateway process (secrets are decrypted in memory)... No HSM/TPM/OS-keyring; no key rotation yet." — TD-15

"Tokens travel in cleartext over HTTP (trusted LAN only until TLS), a single shared operator token (no per-user audit), no lockout." — TD-17

"Nothing was optimised or validated; quarantine is recommended, not enforced (Phases 6-7)." — TD-19

"Nothing in this repository shows that any weight, threshold, half-life, floor, cap or point value was optimised, calibrated or validated: there is no incident dataset, no false-positive/false-negative study, no ROC analysis, no user study." — docs/architecture/trust-engine.md §16

Each of these is treated in this report as a scope boundary, not a flaw to hide — and each becomes a concrete, well-specified next step in §16.

13. External research landscape and novelty analysis

WHAT ALREADY EXISTS

Area	State of the art (2025)
Post-quantum standards	NIST finalised ML-KEM (FIPS 203), ML-DSA (FIPS 204) and SLH-DSA (FIPS 205) in 2024; HQC selected as a fifth backup KEM in March 2025. ML-KEM is already deployed in hybrid TLS 1.3 handshakes in production browsers/CDNs; vendor SDKs (e.g. DigiCert TrustCore) ship FIPS-140-3-certified ML-KEM/ML-DSA for IoT.
Post-quantum device attestation	2025-26 research (e.g. a post-quantum self-healing healthcare-IoT framework, a post-quantum cognitive zero-trust architecture for healthcare IoT) uses ML-DSA/SPHINCS+ to sign checkpoint state hashes and verify checkpoint authenticity before an automated rollback — the same signature-then-recover pattern Q-SHIELD's trust engine specifies (but does not yet enforce) for its own recovery states.
Continuous / dynamic trust scoring	2025 work on continuous trust scoring for IoT sensor streams formulates trust as continuous estimation from rule-derived data-quality proxies over time; ML-driven zero-trust architectures convert intrusion probabilities into dynamic trust scores that drive ALLOW/MONITOR/DENY/QUARANTINE decisions.
Multimodal cyber-physical fusion	2025 research fuses sensor and network-traffic modalities (cross-attention over VAE-LSTM latents; unsupervised multimodal autoencoders) for cyber-physical anomaly detection, and reports containment-action latencies under 300ms in microservice-based IoT security middleware.
Self-healing IoT	2024-25 work explores bio-inspired repair algorithms with explainable GNNs, resilient self-healing intrusion-detection frameworks against DoS/DDoS, and MAPE-K-style fault-tolerance loops for automatic recovery — the general self-healing pattern is active research, mostly at the network or model-update level rather than a per-device cryptographic trust state machine.

WHAT Q-SHIELD IS ACTUALLY DOING, AND WHAT IS / ISN'T NOVEL

None of Q-SHIELD's individual building blocks are novel in isolation: post-quantum KEM/signature schemes are NIST standards used exactly as specified; a weighted multi-factor trust score with caps and decay is a well-known pattern in zero-trust and reputation-system literature; a TRUSTED/SUSPICIOUS/QUARANTINED state machine is a standard security-automation shape. The specification itself does not claim otherwise, and neither does this report.

What this project has actually built that is comparatively unusual for a hackathon-scale prototype is the **integration and the engineering discipline**: a strict separation between authentication and trust (a valid signature is a precondition for evidence, never a positive contribution to score) enforced in code via an explicit authenticity-boundary table per signal kind, not just described in a diagram; a bounded “pressure” term specifically engineered so that unauthenticated network noise cannot by itself cause quarantine (an anti-griefing property many published trust-scoring designs do not address); exact, machine-readable per-event score attribution (every reason sums to the exact delta, tested to 1e-9); and a genuine post-quantum vision-service identity layer wired into that trust pipeline today, not left as future work. The Phase 4.1 self-review — catching and fixing a real design flaw in the recovery state machine before it shipped — is itself evidence of engineering rigor rather than a novelty claim.

WHAT A TECHNICAL JUDGE WOULD CHALLENGE, AND CLAIMS TO AVOID

A judge will likely ask...	Honest answer
"Is the trust engine's math validated against real attack data?"	No — the spec itself says so. It is internally consistent and deterministic, not empirically calibrated.
"Does the ESP32 actually run post-quantum crypto?"	No — HMAC-SHA256 only, by explicit design decision (TD-01).
"Has this been tested on real hardware under a real physical attack?"	No — the firmware has never been compiled or flashed; no physical tamper was ever performed.

A judge will likely ask...	Honest answer
"Is the recovery/self-healing actually automatic?"	No — it is a validated state machine with unit tests; nothing in the gateway drives a device through it or enforces quarantine yet.
"Is this the first system to do continuous IoT trust scoring?"	No — continuous trust scoring for IoT is active 2025 research; the claim to make is the specific combination and engineering, not priority.

DEFENSIBLE NOVELTY CLAIM

"Q-SHIELD's contribution is not a new cryptographic primitive or a new trust algorithm — it is a working, tested integration that keeps authentication and trust strictly separate at the code level, fuses cryptographic identity, device-reported physical/sensor state and post-quantum-signed AI vision evidence into one deterministic and exactly-explainable trust score, and bounds what an unauthenticated attacker can do to that score by construction — built on NIST-standardised post-quantum cryptography (ML-KEM-768, ML-DSA-65) from the outset rather than retrofitted, with a self-healing state machine that is specified, unit-tested, and honestly labelled as not yet enforced."

Sources: Frontiers (post-quantum self-healing healthcare IoT, 2026); PMC (post-quantum cognitive zero-trust architecture for healthcare IoT); MDPI Applied Sciences (continuous trust scoring for IoT sensor streams, 2025); Discover Computing / Springer (cyber-physical anomaly detection via sensor-network fusion, 2026); Frontiers (autonomous cyber-physical security middleware for IoT, 2025); Springer Neural Computing and Applications (self-healing IoT via explainable GNNs, 2026); Nature Scientific Reports (self-healing intrusion detection, 2026); NIST FIPS 203/204/205 (final, 2024) and the March 2025 HQC selection announcement.

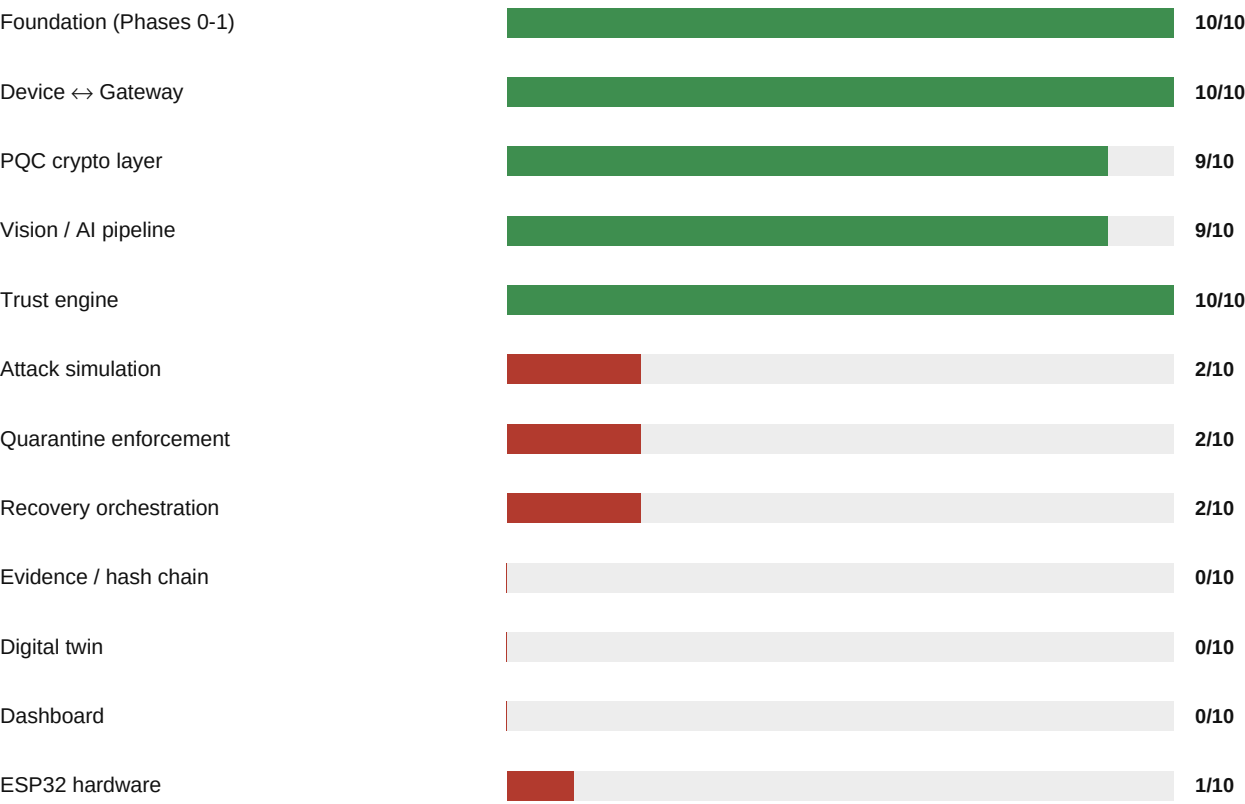
14. Hackathon positioning

Dimension	Assessment
Track fit (Post-Quantum Cryptography)	Strong and precise: ML-KEM-768/ML-DSA-65 are real, correctly used, and tested against NIST ACVP vectors — not a superficial "we used a PQC library once" integration; they gate what trust evidence is accepted.
Problem relevance	Clear and well-articulated: authentication-does-not-equal-trust is a real, recognised gap, and current 2025 research (see §13) is actively working the same problem.
Technical depth	Genuinely deep in two modules (trust engine, PQC/crypto) with 544 passing tests and real benchmarks; shallow or absent in five others (evidence, twin, dashboard, quarantine enforcement, recovery orchestration).
Demonstration potential	Strong for the implemented path: a live gateway, a real webcam, a real forged/replayed attack, a real score drop and state change, shown end to end. Weak for anything past QUARANTINED — there is nothing to demo for enforcement or recovery yet.
Feasibility / scalability	The prototype is single-host, SQLite-backed, and assumes one process owns the database — explicitly documented, not hidden. A real deployment story exists on paper (TD-10's recovery sequence) but is unbuilt.
Societal impact framing	Legitimate but should stay general (industrial IoT, healthcare monitoring, smart infrastructure) rather than claiming any specific deployment, since none has occurred.

This report does not claim judges will select Q-SHIELD. Its strongest demonstrable asset is a real, live, end-to-end run that a judge can watch happen — authenticate, detect, forge, get rejected, watch the trust score fall and the state change, with an exact reason printed for every point lost. Its biggest weakness is the gap between “trust drops and the state changes” and “anything actually happens because of it” — quarantine and recovery are recommendations today, not actions.

15. Final status board

Scored 0–10 per module from repository evidence only (test count and pass rate, presence of a live-run artifact, and whether the module's own documentation claims completeness). Not copied from any template — each score is independently justified in the sections above.



VERIFIED (green, 8–10): trust engine, PQC crypto, vision pipeline, device↔gateway, foundation. PARTIAL/red-low (1–2): attack simulation, quarantine, recovery orchestration, ESP32 hardware. NOT STARTED (red, 0): evidence chain, digital twin, dashboard.

16. Future implementation roadmap

Ordered by dependency, not by wish — each later phase genuinely depends on the one before it, mirroring TD-10's own recovery sequence.

IMMEDIATE (highest demo value, lowest complexity)

Feature	Purpose	Complexity	Required for hackathon demo?
Attack-simulation suite	Script 4–6 more real scenarios (tamper, sensor out-of-range, config-integrity mismatch, rogue/unknown device) the way demo_forger.py already does one	Low — extends an existing, working pattern	Yes — directly strengthens the live demo
Quarantine enforcement (minimal)	When state == QUARANTINED, actually reject non-recovery endpoints for that device instead of only recommending it	Low-medium — one gateway-side check against the trust service	Strongly recommended — closes the most visible gap

NEXT

Feature	Purpose	Complexity	Depends on
Recovery orchestrator	Drive a quarantined device through RECOVERING→VERIFIED→RECOVERED against real health checks (TD-10's sequence)	Medium-high	Quarantine enforcement; a defined set of health checks
Evidence hash chain	Append-only, ML-DSA-signed log of trust/security events for after-the-fact proof (TD-09's design)	Medium	None blocking — could start now
Minimal dashboard	Read-only view of device status, trust score/history, and the event stream already exposed by GET /api/v1/trust/*	Medium	None blocking — the API already exists

ADVANCED

Feature	Purpose	Research value
Digital twin comparison module	Turn the two existing config fields (expected fw/config hash) into an actual twin-state store with drift detection	Moderate — a well-trodden pattern applied to this specific trust engine
Real ESP32 hardware	Compile, flash, wire a real tamper switch and sensors; replace device_agent with genuine telemetry	High demo value; the biggest single credibility gap today
Parameter calibration	Replace the trust engine's unvalidated design-choice parameters (§5.7 category C/D) with values tuned against real incident or false-positive data	High research value — this is where a real contribution to the trust-scoring literature could come from

FUTURE RESEARCH

External audit of the custom ML-KEM+HKDF+AES-GCM session protocol; a documented accuracy study for the vision pipeline; multi-gateway / multi-process coordination (today's trust service assumes one process owns the database); formal analysis of the trust-score formula's resistance to adversarial gaming beyond the pressure cap already in place.

17. Team pitches and the live-demo flow

HOW TO EXPLAIN Q-SHIELD TO A TEAMMATE

10-second pitch

"We don't just check if a device is who it says it is once — we keep checking if it can still be trusted, using post-quantum crypto."

30-second pitch

"Every device message and every AI camera observation gets cryptographically verified — the camera path with post-quantum ML-DSA-65 signatures. That verified evidence feeds a trust engine that scores six things: identity, physical tamper, config integrity, sensors, vision and network liveness. One bad signal nudges the score down; two confirmed together — like tamper plus a camera detection — forces quarantine no matter what the rest of the score looks like. We proved it live: forged and replayed an observation against our own gateway and watched the score fall from 100 to 73 with an exact reason for every point."

1-minute pitch

Add: "What's real today: the gateway, the trust engine, the PQC crypto, and the vision pipeline — 544 passing tests, real benchmarks, a real live run. What's not built yet: actually enforcing quarantine (right now it's a recommendation), automatically driving recovery, an evidence hash-chain for forensics, a digital twin, and a dashboard. We're not hiding that — it's the honest next-phase roadmap, and the trust engine and state machine were already built to support all of it."

3-minute pitch

Walk the architecture diagram in §3 top row, then the state-machine diagram in §5.4, then the worked example in §5.5 exactly as it happened on the live gateway. Close with the honest gap: "we validated the hard part — deciding, correctly and explainably, when a device should no longer be trusted. Acting on that decision is the next phase, and we know exactly what it needs because we already designed and unit-tested the state machine for it."

WHAT HAPPENS DURING OUR LIVE DEMO

1. Start the real gateway process against a scratch database. 2. Enroll DEVICE-001, provision its PQC signer. 3. Send authenticated telemetry — show the trust API report score 100, TRUSTED. 4. Run the real webcam + YOLO11n pipeline live, signing observations over an ML-KEM-768 session — show a genuine detection lower the score slightly, still TRUSTED. 5. Run scripts/demo_forgery.py against the live gateway — show the 401 (forged signature) and 409 (replay) rejections in the terminal. 6. Query the trust API again — show the score at 73, state SUSPICIOUS, and the exact itemised reason for every point lost. 7. Close with the state-machine diagram: "here is exactly what would happen next if we wired up enforcement — and here is the unit-tested logic already waiting for it."

This document was produced by inspecting the Q-SHIELD repository's actual source code, its 544-test suite, and the JSON artifacts of its own live runs and benchmarks as the sole source of truth. No feature was upgraded from planned to implemented because scaffolding existed; no accuracy or performance number was invented; every quoted limitation comes from the project's own documentation.